

# StampFly で学ぶドローン制御

## 高校の先生向け体験講座 (DXH)

伊藤 恒平

2026 年 7 月 18 日 (土) ・ 19 日 (日)



本日のスライド資料

## 伊藤 恒平 — 金沢工業大学 ロボティクス学科 教員

StampFly 開発者 / 専門はドローン制御工学

---

経歴 大学で航空宇宙工学を学ぶ  
陸上自衛隊に入隊、飛翔体の研究開発に従事  
高専教員を経て、金沢工業大学 ロボティクス学科  
教員

---

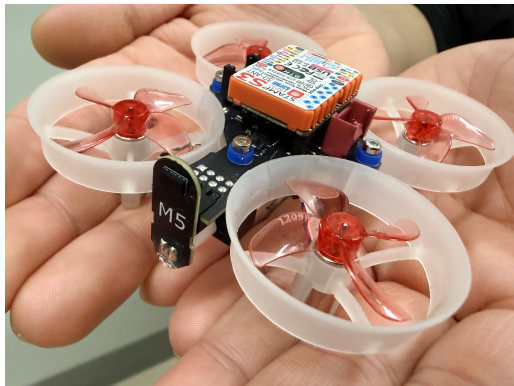
趣味 ロボット作り・読書・映画鑑賞

---

# 本日の流れ

| 時間                   | 内容                 |
|----------------------|--------------------|
| 13:30 – 13:40 (10 分) | オープニング             |
| 13:40 – 14:05 (25 分) | ① 操縦体験 (シミュレータ→実機) |
| 14:05 – 14:30 (25 分) | ② 開発環境+書き込み体験      |
| 14:30 – 15:15 (45 分) | ③ プログラム書き換え・モータの制御 |
| 15:15 – 15:30 (15 分) | まとめ                |

# StampFly とは

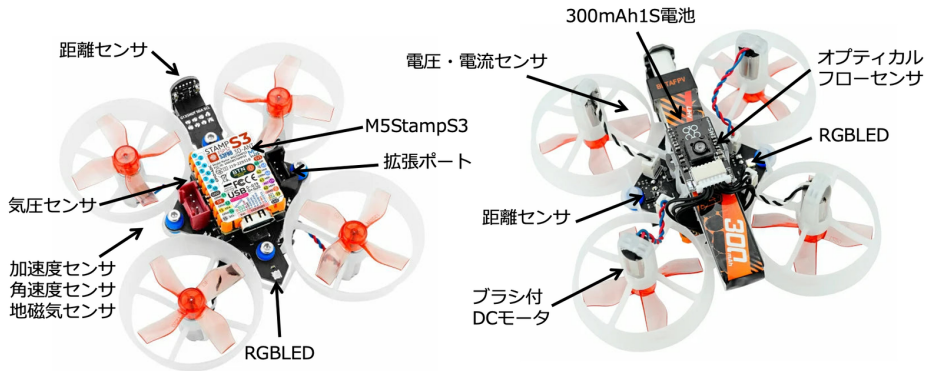


| 項目    | 仕様                    |
|-------|-----------------------|
| MCU   | ESP32-S3 (M5Stamp S3) |
| IMU   | BMI270 (6 軸 400 Hz)   |
| 気圧    | BMP280                |
| ToF   | VL53L3CX              |
| 質量    | 37 g (バッテリー含む)        |
| 通信    | ESP-NOW + WiFi        |
| バッテリー | LiPo 1S 3.7 V         |

StampFly は **M5Stack 社 CEO の Jimmy 氏**と講師の**共同開発**です

# StampFly の搭載センサ・部品

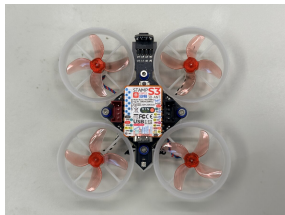
## StampFly



# StampFly と Tello の違い



Tello EDU (約 87 g ・ カメラ搭載)



StampFly (37 g)

|         | Tello                              | StampFly                 |
|---------|------------------------------------|--------------------------|
| プログラミング | 飛行コマンドを外から送る<br>(Scratch / Python) | 機体内部の制御プログラム自体<br>を書き換える |
| ソフトの中身  | 非公開 (変更できない)                       | 全ソースコード公開                |
| センサのデータ | SDK 経由の一部のみ                        | 生データに直接アクセス可             |

今日の③「モータの制御」は、中身を書き換えられる StampFly ならではの体験

# StampFly の現在地と、今日の楽しみ方

## StampFly はいま、育ちざかりの教材です

位置制御まで飛ぶようになり、教材の土台が整ってきました。  
ここから「授業でそのまま使えるパッケージ」へ仕上げていく段階です

- 今日は、完成すると見えなくなる開発の生の姿（コマンド・ビルド・調整の途中）まで体験できます
- ドローンの「中身」に手を入れられるのは、いまの StampFly ならではの楽しみです

## 目指しているもの

初等教育から高等教育・社会人教育までを見据えた教材群の完成 — 今日はその種まきに、皆さんをお招きしています

# ドローンは「勝手に」飛んでいるのではない

## Tello はなぜ「言うことを聞く」のか

「前に進め」で前に進み、「回れ」で回る — それは膨大な**試行錯誤**（制御工学）の賜物です。

素のドローンは、制御なしでは**1秒も浮いていません**

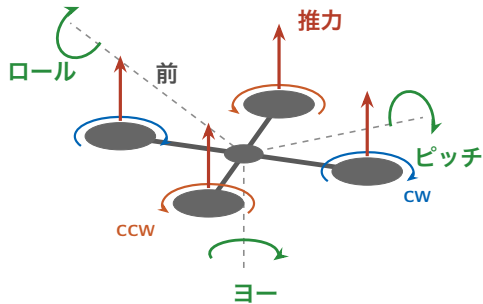
- クアッドコプタは本質的に不安定 — 放っておくと勝手に傾いて墜落する
- センサで姿勢を測り、4つのモータを絶えず調整し続けて、はじめて浮いている

## だから、この教材は「制御」から始まります

**飛ばないドローンを、制御で飛ばせるようにする** — その中身を生徒と一緒に開けて学べる教材を、StampFly は目指しています



# クアッドコプタはどうやって飛ぶのか



| 動き         | 作り方                      |
|------------|--------------------------|
| 浮く・上下      | 4つの推力の合計を増やす／減らす         |
| ロール（左右に傾く） | 左右の推力の差                  |
| ピッチ（前後に傾く） | 前後の推力の差                  |
| ヨー（機首を回す）  | CW組とCCW組の回転数の差<br>（反トルク） |

傾けば、傾いた方向へ進む。この推力配分を **1秒に400回**自動調整し続けるのが「制御」

# 今日つかう道具一式 (StampFly エコシステム)



ソースコード・教材・シミュレータはすべて Web で無料公開  
学校でも今日と同じ環境をそのまま用意できる

# 安全ルール：飛行の安全

## 飛行前の確認

- 保護メガネを全員着用
- 飛行エリアは区画・ネットで分離
- プロペラガードの状態を確認
- 長い髪はまとめる
- 機体の真上に顔を出さない

## 緊急停止

**DISARM** = モータを止める操作。飛行中は自動着陸 → もう一度で即時停止

## バッテリー

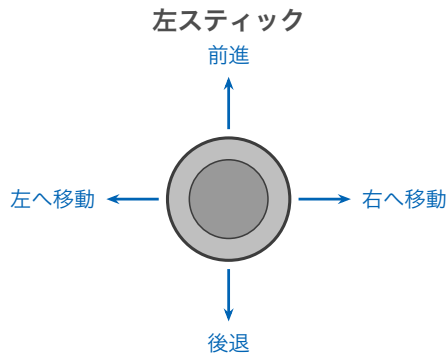
満充電 4.2V / 3.4V で低電圧警告。膨らんだ電池は使用しない。充電中は目を離さない

①

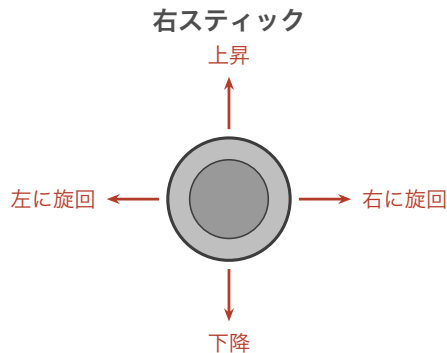
**操縦体験（シミュレータ→実機）**

StampFly DXH ワークショップ

# ① 操縦のしかた（スティックモード 3）



（機体の水平移動）



（高度と機首の向き）  
押し込み = ARM / DISARM

今日つかう飛行モード **ALT\_HOLD**（高度維持）では高度は自動維持 — 操縦は主に**左スティック**を少しずつ倒す

# ① まずはシミュレータで飛ばしてみる

コマンドプロンプト（文字で PC を操作する画面）で実行

```
sf sim run ← 3Dシミュレータが起動
```

※「←」以降は説明なので入力しません（sf = StampFly 用コマンド）

- コントローラを USB ケーブルで PC に接続（設定済み）
- 起動直後の校正が終わるまでスティックに触らない
- スティックの割り当ては**実機とまったく同じ**



シミュレータ画面 — 画面の StampFly を操縦する

# ① シミュレータ → 実機への切り替え

## コントローラを実機用（ESP-NOW）に戻す

- ① コントローラの **M5 ボタン**（画面下）でメニューを開く
- ② 「**通信モード**」を**1 回選ぶ** → USB HID から ESP-NOW（ルーターなしで機体と直接つながる無線）に切り替わり、自動で再起動する
- ③ USB ケーブルを外す（実機とは電波でつながる）

※ 通信モードを切り替えても、ペアリングなどの設定はそのまま残ります

# ① 機体の起動と LED の見方

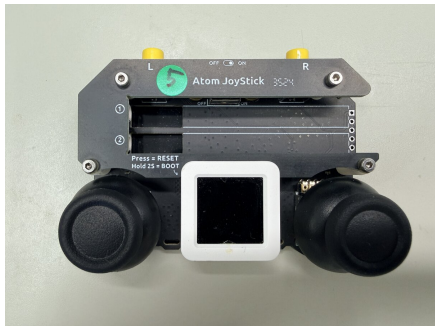
| LED・音         | 状態               |
|---------------|------------------|
| 白色点灯 + 起動音    | 起動中              |
| マゼンタ点滅        | ジャイロ校正中（機体に触らない） |
| 緑色常灯 + ピッ × 3 | 準備完了（約 10 秒）     |

## 起動時の注意

平らな床に置いて電源を入れる。校正が終わる（緑常灯）まで機体を動かさない



# ① コントローラのボタンとスティック



Atom JoyStick (左上が L ボタン)

| 飛行モード     | 機体 LED 色 |
|-----------|----------|
| STABILIZE | 黄緑       |
| ACRO      | 青        |
| ALT_HOLD  | オレンジ     |
| POS_HOLD  | マゼンタ     |

## ALT\_HOLD への切り替え

L ボタンを **1回** 押す (OFF → ALT\_HOLD)  
機体 LED が**オレンジ**になったことを確認する

# ① ALT\_HOLD と自動離着陸

## 自動離着陸と ARM / DISARM

**ARM**（右スティックを押し込む）→ 自動離陸・高度 0.5 m を自動維持

**DISARM**（もう一度押し込む）→ 自動着陸 → 着陸中にもう一度で緊急停止

## 低電圧警告

シアン点滅 + 警告音 → 着陸して電池交換（飛行時間 約 4 分）

## 進め方

1 人 1 セット・全員同時に飛行（交代なし）。ペアリングは 1 対 1 — 隣の機体は反応しません。電池が切れたら（シアン点滅）交換して再開

②

## 開発環境 + 書き込み体験

StampFly DXH ワークショップ

## ② 開発環境の全体像

### 今日は構築済み

貸出 PC には ESP-IDF（組み込み開発環境）と sf CLI をセットアップ済み。  
実習は書き込みからスタートします

```
git clone https://github.com/M5Fly-kanazawa/stampfly_ecosystem
cd stampfly_ecosystem
install.bat      ← ESP-IDF + sf CLI を自動インストール
setup_env.bat   ← 開発環境をアクティベート
```

上記は「自分の PC でも試したい」ときの手順（今日は実施不要）。「←」以降は説明なので入力しません

### このあと

GUI フラッシャ（デスクトップアプリ）のデモも行います

## ② 書き込みの仕組み — 3つの方法

### ファームウェアとは

機体を動かすプログラム一式のこと。「書き込み」とは、新しいファームウェアを StampFly の中に転送する作業を指す

|        | Web フラッシュャ            | GUI フラッシュャ     | sf CLI (コマンド)    |
|--------|-----------------------|----------------|------------------|
| 起動方法   | ブラウザ<br>(Edge/Chrome) | デスクトップア<br>プリ  | コマンドプロンプト        |
| 書き込むもの | 公開されている完<br>成版        | 公開されている完<br>成版 | PC 内でビルドした<br>もの |
| 本日の用途  | 紹介のみ                  | デモのみ           | ②・③の実習           |

今日の実習は②・③とも sf CLI で書き込みます。Web / GUI フラッシュャは開発環境のない PC でも使える方法（紹介のみ）。

※ ビルド＝人が書いたコードを機体が実行できる形式に変換する作業

## ② 実習：sf flash でファームウェアを書き込む

### PC 側の操作（コマンドプロンプト）

```
cd stampfly_ecosystem
setup_env.bat
sf flash vehicle    ← 機体をUSB接続してから実行
```

※「←」以降は説明なので入力しません

- ① 機体の電源を入れ、USB-C ケーブルで PC に接続
- ② 上のコマンドを実行（PC 内でビルド済みの標準ファームウェアを書き込み）
- ③ 完了後、機体が再起動 — 起動音が終わり LED が緑常灯になるまで水平静置（ジャイロ校正）

## ② よくある失敗と対処

| 症状              | 対処                                          |
|-----------------|---------------------------------------------|
| 機体が認識されない/繋がらない | ケーブルを交換（充電専用ケーブルは不可）                        |
| 書き込みが進まない/失敗する  | ポートを使う他アプリ（シリアルモニタ等）を閉じる                    |
| それでも失敗する        | 機体の BOOT ボタンを押しながら USB を差し直して、もう一度 sf flash |

※ sf flash は機体の設定（ペアリング等）には触れません。失敗したら、そのままもう一度実行するだけです

※ BOOT ボタン＝機体上面 M5StampS3 のボタン（③で ARM に使うものと同じ。起動時に押しながらだと書き込みモード）

③

## プログラム書き換え・モータの制御

StampFly DXH ワークショップ



### ③ モータ実習の安全

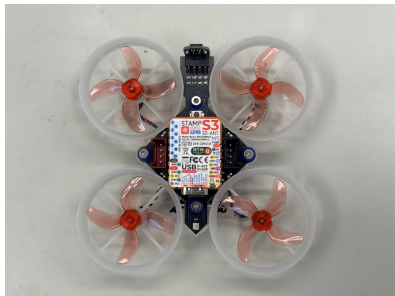
#### プロペラを回す実習ならではの注意

- **Duty は 0.15 以下**（ファームでは強制されないので数値を必ずダブルチェック）
- プロペラは付けたまま — 機体上部の基板を上から**指で押さえて**回す
- バッテリーと USB の同時接続は避ける（動作確認はバッテリー駆動）

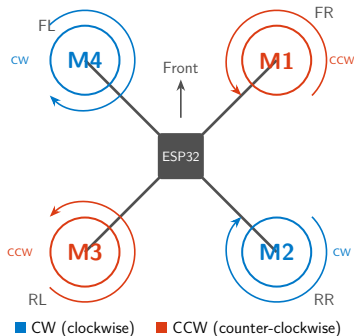
#### 止め方

機体本体ボタンをもう一度クリック（DISARM）／完全に切るときはバッテリーを外す

### ③ モータ配置

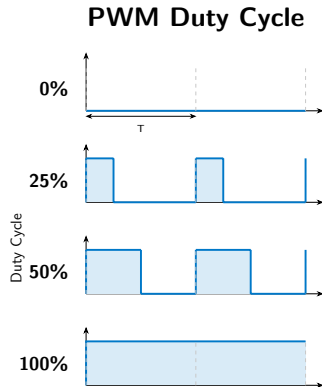


実機（上から見た写真）



- モータ ID : 1 = FR (右前) / 2 = RR (右後) / 3 = RL (左後) / 4 = FL (左前)
- 対角のモータが同じ方向に回転 (トルクバランス) :  $M1 \cdot M3 = \text{CCW}$  /  $M2 \cdot M4 = \text{CW}$

### ③ PWM と Duty / ARM・DISARM とは



#### PWM とは

- モータの力加減は、電源を**高速で ON/OFF** して調整する (PWM)
- ON の時間の割合が **Duty** — 0% = 停止、50% = 半分の力、100% = 全力
- 身近な例: LED 照明の明るさ調整も同じ仕組み

#### ARM / DISARM とは

- **ARM** = モータ出力を有効化 (回転可能に)
- **DISARM** = モータ出力を無効化 (強制停止)

### ③ 実習の手順 (1) : ビルドと書き込み

#### PC 側の操作 (コマンドプロンプト)

```
cd stampfly_ecosystem
setup_env.bat
sf lesson edit      ← VSCodeが開く。コードを編集して保存
sf lesson build
sf lesson flash     ← 書き込み後シリアルモニタが自動起動
```

※ 「←」以降は説明なので入力しません (打ち込むのはコマンドだけ)

シリアルモニタの終了は Ctrl+]

## ③ 実習の手順（2）：機体側の動作確認

### バッテリー駆動で確認する

- ① USB ケーブルを外す
- ② バッテリーを接続して機体の電源を入れる
- ③ 起動 LED シーケンスを待つ（右表）
- ④ 機体本体ボタンを **1回クリック** → ARM（設定した Duty で回転）
- ⑤ もう一度クリック → DISARM（停止）

| # | LED           | 状態                |
|---|---------------|-------------------|
| 1 | 白（明滅）         | 置いて待つ（約 3 秒）      |
| 2 | 青（明滅）         | センサ初期化中           |
| 3 | マゼンタ（点滅）      | 安定化待ち（約 3 秒・触らない） |
| 4 | 緑点灯+ビープ<br>×3 | 操作可能              |

※ 実習用ファームの表示です（①の機体とは異なります）

## 安全

プロペラは付けたまま — 機体上部の基板を上から指で押さえて回す

## ③ 実習コード

user\_code.cpp

```
1  #include "workshop_api.hpp"
2
3  // =====
4  // Motor duty test (hardcoded)
5  // 各モータの Duty をハードコードする
6  // =====
7  //
8  // Motor IDs / モータ ID:
9  //   1 = FR (右前)   2 = RR (右後)
10 //   3 = RL (左後)   4 = FL (左前)
11 //
12 // Duty range: 0.0 - 1.0
13 //   0.0 = stop / 止める
14 //   0.10 = spin at 10% / 10% で回す
15 //   このチュートリアルでは 0.15 を上限とする
16
17 void setup()
18 {
19     ws::print("Motor duty test");
20
21     // Do NOT auto-arm in code. Arming is controlled by the controller's
22     // ARM button or a single click of the vehicle's onboard button, so
23     // motors never spin until you intentionally press it.
24     // コードでは自動 ARM しない。アーム/解除はコントローラの ARM ボタン
25     // または機体本体ボタンの単クリックで行うので、意図的に押すまで
26     // モータは絶対に回らない
27 }
28
29 void loop_400Hz(float dt)
30 {
31     // ----- Set each motor's duty here / 各モータの Duty をここで設定 -----
32     ws::motor_set_duty(1, 0.10f); // FR (右前)
33     ws::motor_set_duty(2, 0.00f); // RR (右後)
34     ws::motor_set_duty(3, 0.00f); // RL (左後)
35     ws::motor_set_duty(4, 0.00f); // FL (左前)
36     // -----
37 }
```

### ③ 改造課題：推力の差で「動き」を作る

#### 4つの Duty に差をつけて ARM してみる

飛行原理のページで見た「推力の差 → ロール・ピッチ・ヨー」を、机の上で手に感じる実習です

| 設定 (Duty)                                   | 機体が「しようとする」動き       |
|---------------------------------------------|---------------------|
| 右 2 つ ( $M1 \cdot M2$ ) = 0.10、左 2 つ = 0.05 | 左に傾こうとする (ロール)      |
| 前 2 つ ( $M1 \cdot M4$ ) = 0.10、後 2 つ = 0.05 | 後ろに傾こうとする (ピッチ)     |
| CW 組 ( $M2 \cdot M4$ ) = 0.10、CCW 組 = 0.05  | 機首が回ろうとする (ヨー・反トルク) |

#### 必ず守る

機体上部の基板を上から指で押さえたまま ARM する。傾く「力」を感じたら DISARM

### ③ チェックポイント

#### 確認事項

- ☐ Duty を書き換えてビルド・書き込みができた
- ☐ 機体本体ボタンで ARM → モータ回転 → DISARM → 停止を確認した
- ☐ 指定したモータが指定の Duty で回ることを確認した



# まとめ

## 授業導入に向けて

StampFly DXH ワークショップ

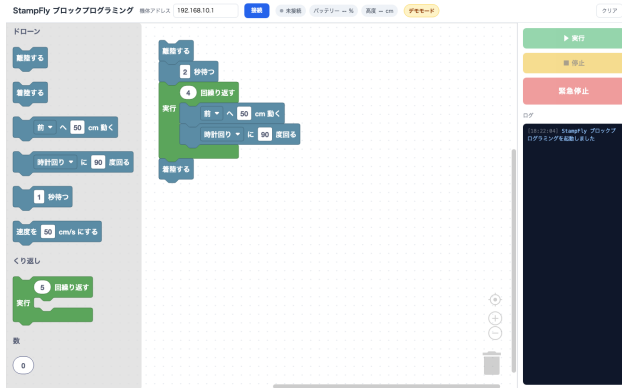
# これからの StampFly (1) : Python から操縦

- Tello SDK 互換の操縦 API を実装  
(開発中・実機検証中)
- Tello 用の Python ライブラリ  
(djitellopy) のプログラムが**そのまま動く**ことを目標
- Tello で作った授業教材・生徒の  
コードを StampFly に載せ替えら  
れる

## tello\_compat.py

```
1 from djitellopy import Tello
2
3 drone = Tello()
4 drone.connect()
5 drone.takeoff()
6 drone.move_forward(50)
7 drone.rotate_clockwise(90)
8 drone.land()
```

# これからの StampFly (2) : ブロックプログラミング



- Google の **Blockly** — ブロックを組み合わせて書くビジュアルプログラミング言語
- ブラウザでブロックを並べて飛行プログラムを作れる（開発中・デモ版が動作）
- キーボード入力なしで、プログラミング未経験の生徒でも扱える

sf blocks — ブラウザのブロック編集画面（デモ版）

# StampFly の入り口 — Landing ページ



[https://m5fly-kanazawa.github.io/stampfly\\_ecosystem/](https://m5fly-kanazawa.github.io/stampfly_ecosystem/)

- StampFly エコシステムの紹介ページ
- ドキュメント・教材への入り口
- Web フラッシュもここから — ブラウザだけで最新ファームウェアを書き込める（開発環境不要）

学校の PC に開発環境がなくても、このページから書き込みを試せます。今日の続きはここから

# おわりに — 現在地と、これから

## StampFly はここまで来ました

位置制御まで飛ぶようになり、「授業でそのまま使えるパッケージ」へ仕上げていく段階です。

制御の完成度を上げる開発を続けています

- 種はすでに撒かれています: 全ソース公開・シミュレータ・Tello 互換・ブロックプログラミング、そして今日の講座

## 目指す姿

**初等教育から高等教育・社会人教育まで、StampFly を教材にした一貫した教材群を完成させる**

— 皆さんの声（アンケート）が、その羅針盤になります

# アンケートのお願い（2種類） / リンク集

大学アンケート  
（DXH 事業）  
QR（当日掲示）

講師アンケート  
（教材改善用）  
QR（当日掲示）

2 種類のアンケートにご協力をお願いします

## GitHub リポジトリ

[https://github.com/M5Fly-kanazawa/stampfly\\_ecosystem](https://github.com/M5Fly-kanazawa/stampfly_ecosystem)

## 本日のスライド資料 （ドクセル）

[https://www.docswell.com/s/Kouhei\\_Ito/ZL3EYL-2026-07-17-161742](https://www.docswell.com/s/Kouhei_Ito/ZL3EYL-2026-07-17-161742)

# 本日はありがとうございました